1 Choreographer

1.1 System overview

Native Rust development profile for local Java source `Choreography/`. One independently configured executable, identity, port and state file per role. No Java, Go or Python process implements the service. Some essential operations are explicitly unavailable.

1.2 Design and trust boundaries

Tokio/Hyper/Axum with rustls enforces mutual TLS. Certificate chain, validity, local CN/profile and startup-configured optional CRLs are checked. CRLs need process restart to refresh; without CRLs no revocation enforcement is claimed. Tokens are verified with Authentication and bound to the TLS identity. Business calls require ConsumerAuthorization; management requires a verified sysop or explicit broad whitelist. CA provisioning instead applies Java-style named certificate-principal rights. Outbound authorities are pinned to expected CN/profile before credentials are sent, in addition to chain, SAN and time validation.

1.3 Persistent state

Independent locked atomic JSON replaces SQL. Clone/validate/fsync/rename commits prevent partially published local transactions. Rename is the commit point; directory-sync failure is an uncertain durable commit, not a rollback. Owner-only keys/password files are separate. Network effects are not distributed transactions; interrupted starts and bridge operations require reconciliation. No automatic Java state import, clustering or deployment is delivered.

1.4 Implemented behavior

Plan/executor CRUD, single local action/step starts, aborts and completion callbacks

1.5 Open release gates

Dependencies, parallel/multi-action execution, quantities, optimal selection, auto discovery, notifications, async orchestration, exact executor DTO parity. All systems remain development profiles, not certified interchangeable Java replacements. See `COMPATIBILITY.md` for detailed DTO fields and restrictions; the Java sources remain the authoritative contract.

1.6 Produced and consumed services

Advertised implemented HTTPS definitions: `choreographerExecutor`, `choreographerManagement`, `choreographerSession`, `echo`, `generalManagement`.

Authorities consumed as applicable: Authentication, ServiceRegistry, ConsumerAuthorization and explicitly configured pinned adapter/provider origins. Only implemented operations are registered. Service versions are 1.0.0 except Gatekeeper/GateTunnel 5.2.2. No AMQP sysMonitor.observe implementation is advertised.

1.7 HTTPS operations

JSON unless raw configuration bytes, PEM CRL or echo text. Bodies: 1 MiB; remote responses: 8 MiB. Header: `Bearer IDENTITY-TOKEN//token`. Unknown routes return 404, absent dependencies 503. Unported

operations return 501 after applicable access checks and are excluded from registration. Route presence is not semantic parity.

Method	Path	Access / status
GET	/choreographer/general/mgmt/get-config	management
POST	/choreographer/general/mgmt/logs	management
GET	/choreographer/monitor/echo	certificate
POST	/choreographer/executor/register	identity
DELETE	/choreographer/executor/unregister	identity
POST	/choreographer/executor/notify-step-done	identity
POST	/choreographer/session/start	identity
DELETE	/choreographer/session/abort/{id}	identity
POST	/choreographer/orchestration-result/{correlationId}	identity / 501
GET	/choreographer/mgmt/plans	management
POST	/choreographer/mgmt/plans	management
GET	/choreographer/mgmt/plans/{id}	management
DELETE	/choreographer/mgmt/plans/{id}	management
GET	/choreographer/mgmt/plans/{id}/check	management
POST	/choreographer/mgmt/sessions/start	management
DELETE	/choreographer/mgmt/sessions/{id}	management
GET	/choreographer/mgmt/sessions	management
GET	/choreographer/mgmt/session-steps	management
GET	/choreographer/mgmt/worklog	management
GET	/choreographer/mgmt/executors	management
POST	/choreographer/mgmt/executors	management
GET	/choreographer/mgmt/executors/{id}	management
DELETE	/choreographer/mgmt/executors/{id}	management

Error envelope: errorMessage, errorCode, exceptionType, origin and timestamp. Full Java status, DTO and validation equivalence requires differential testing.

2 References and qualification

See `contracts.json` for local Java SHA256 hashes/mappings, `COMPATIBILITY.md` for DTO details and gaps, and `VERIFICATION.md` for tests actually run. No upstream release-tag provenance, complete replacement or real-cloud interoperability is asserted.